

Low-Level Design

Prior Authorization Multi-Agent Platform

**DEMONSTRATION SYSTEM — SYNTHETIC DATA — NOT FOR
CLINICAL
OR COVERAGE USE**

Document type: Low-Level Design (LLD)

Version 1.1

Audience: engineers implementing, extending, or maintaining the system

Companion document: High-Level Design (HLD)

1. Module & Package Structure

1.1 Backend (backend/app/)

app/	
api/	health, patients, guidelines, search, orchestration, review, observability
core/	config.py (settings incl. LLM_PROVIDER + both provider configs), logging.py, exceptions.py
db/	models.py (SQLAlchemy ORM), database.py (engine/session), repositories.py
schemas/	Pydantic request/response models (patient, guideline, orchestration, response, review, observability)
agents/	base.py (BaseAgent), intake_agent, retrieval_agent, risk_agent, criteria_matching_agent, determination_agent, safety_agent
orchestration/	state.py, workflow.py, orchestrator.py, determination_rules.py
rag/	embeddings.py, chunker.py, faiss_store.py, ingestion.py, retriever.py
services/	orchestration_service, review_service, metrics_service, llm_client, report_service, patient_service, guideline_service
utils/	text.py, validators.py

1.2 Frontend (frontend/src/)

src/	
api/	client.ts (typed REST client), types.ts (mirrors backend schemas)
agentPipeline.ts	live pipeline step defs + pure state-transition helpers
charts/	colors.ts, ChartTooltip.tsx, StackedOutcomeBar.tsx, AgentLatencyChart.tsx, AgentReliabilityChart.tsx, CostTrendChart.tsx
components/	Badge.tsx, SafetyBanner, AssessmentResult, AgentPipeline, AgentTraceView, PatientForm
pages/	PatientsPage, PatientDetailPage, GuidelineSearchPage, ReviewQueuePage, ReviewDetailPage, ObservabilityPage
App.tsx, main.tsx, index.css	

2. Data Model

SQLite via SQLAlchemy 2.0 ORM. Six tables:

Table	Key columns	Purpose
patients	id, patient_identifier (unique), age, gender, vitals/labs, medical_history (JSON), current_medications (JSON), allergies (JSON)	Synthetic patient records.
guidelines	id, guideline_id (unique), title, organization, version, condition, source_file	One row per ingested synthetic payer policy document.

Table	Key columns	Purpose
guideline_chunks	id, guideline_id (FK), chunk_id (unique), text, section, page, metadata_json	Chunked policy text; mirrored 1:1 into the FAISS index by insertion order.
orchestration_runs	id, patient_id (FK), requested_service, service_category, diagnosis_codes (JSON), urgency, prior_treatments_tried (JSON), status, determination, decision_rationale, final_response_json, review_status, reviewer_name, reviewer_decision, final_determination, reviewer_notes, reviewed_at, created_at, completed_at	One row per PA request/run — both AI-processing state and human-review state live here (two separate dimensions).
agent_outputs	id, run_id (FK), agent_name, status, input_json, output_json, execution_time_ms, created_at	Full per-agent trace: one row per agent execution per run.
criteria_evaluations	id, run_id (FK), criterion, status, patient_evidence, guideline, section, source, created_at	Durable, independently queryable audit trail of every criterion evaluated (separate from the JSON blob).

2.1 Notes on orchestration_runs

- **status** (running / completed / insufficient_information) tracks whether the agent pipeline finished; it is set once, synchronously, within the request/stream call.
- **review_status** (pending_review / reviewed) is a separate dimension: whether a human has signed off. Every completed run starts as pending_review.
- **determination** is the AI's decision; **final_determination** is what a reviewer confirmed (equal to determination on uphold, reviewer-chosen on override).
- **final_response_json** stores the complete FinalClinicalResponse payload exactly as produced at completion time, so the review screen and the PDF report always reflect what was actually decided, not a re-computation.

3. Agent Design

All agents implement **BaseAgent.execute(state)**, which times the call, catches any exception (recording status="error" instead of propagating), and appends an AgentTrace to the shared ClinicalWorkflowState. Agents never crash the workflow.

3.1 clinical_intake_agent

Computes BMI if missing (from weight/height). Builds a list of human-readable extracted facts. Validates required clinical fields and flags a request as missing "diagnosis code(s)" if none were supplied.

3.2 guideline_retrieval_agent

Builds a retrieval query from requested_service + diagnosis_codes + clinical_question + patient medical_history + prior_treatments_tried. Embeds it (off the event loop via asyncio.to_thread — see §7.3), searches FAISS with top_k (default 8), and discards results below MIN_RELEVANCE_SCORE = 0.15.

3.3 risk_stratification_agent

Deterministic threshold-based classification (blood pressure, lipids, glycemic status, BMI, smoking, cardiovascular history) into low / moderate / high, used as contextual signal for the reasoning/rationale steps.

3.4 criteria_matching_agent

The core evaluation engine. Key steps:

1. Restricts evaluation to the single **dominant guideline**: the guideline_id whose best-scoring retrieved chunk has the highest score, so unrelated policies never contribute criteria to this request.
2. Extracts every "Criterion N: ..." sentence from the dominant guideline's retrieved chunks via regex.
3. Classifies each criterion through ordered tiers and evaluates it:

Tier	Trigger	Evaluation
Numeric (BMI)	Criterion text contains "BMI"	Parses threshold number(s) from the text and compares directly against the patient's actual BMI value → MET / NOT_MET / UNKNOWN (if BMI undocumented).
Exclusion (negated)	Text matches an exclusion-hint phrase (e.g. "is not considered")	Never asserts NOT_MET from lexical overlap. Overlap ≥ 2 tokens → UNKNOWN (flag for reviewer); otherwise not surfaced.
Step-therapy / documentation	Text matches a step-therapy hint (e.g. "conservative therapy", "documented participation")	MET if the request's prior_treatments_tried overlaps the criterion text; otherwise UNKNOWN (never NOT_MET — absence of documentation is a gap, not a denial).
Default / general	Everything else	Token overlap ≥ 2 with patient/request facts → MET; overlap == 1 → UNKNOWN; 0 → not surfaced.

3.4.1 Design note: two correctness fixes found via live testing

(1) BMI criteria are alternative qualifying bands (e.g. "BMI ≥ 40" OR "BMI 35-39.9 with a comorbidity"), not a checklist every band must pass — see determination_rules.py, §4. (2) The exclusion tier was originally allowed to assert NOT_MET from overlap, which could confidently deny a patient who actually satisfied the negated condition; it now only ever produces UNKNOWN. Both are covered by dedicated regression tests.

3.5 determination_agent

Deterministically decides the outcome, then optionally drafts the rationale via the configured LLM provider:

```
not_met = blocking_not_met_criteria(criteria) # BMI-aware, see below
if not_met:
    determination = "denied"
elif any UNKNOWN:
    determination = "pending"
elif any MET:
    determination = "approved"
else:
    determination = "pending"

rationale = llm_client.generate_determination_rationale(
    requested_service, determination, criteria_evaluated, patient_summary,
) # Anthropic or OpenAI (per LLM_PROVIDER) explains the decision; it never
# changes it. Falls back to a deterministic template if no key is
# configured for the selected provider, or the call fails.
```

Also runs a simple allergy/current-medication substring overlap check and appends a safety flag if one is found.

3.6 safety_validation_agent

Independently re-derives the expected determination from the same criteria (via the shared determination_rules.expected_determination()) and flags a mismatch; verifies every cited criterion string is

actually a substring of some retrieved chunk's text; flags missing information and high overall risk; always sets `requires_clinician_review = True`.

4. Shared Decision Rules (`determination_rules.py`)

Both the determination agent (to decide) and the safety agent (to independently re-check) call the same two functions, so they can never silently diverge on what "not met" means:

```
def blocking_not_met_criteria(criteria):
    bmi_has_met = any(
        "bmi" in c.criterion.lower() and c.status == "MET"
        for c in criteria
    )
    return [
        c for c in criteria if c.status == "NOT_MET"
        and not (bmi_has_met and "bmi" in c.criterion.lower())
    ]

def expected_determination(criteria):
    not_met = blocking_not_met_criteria(criteria)
    if not_met:
        return "denied"
    if any(c.status == "UNKNOWN" for c in criteria):
        return "pended"
    if any(c.status == "MET" for c in criteria):
        return "approved"
    return "pended"
```

5. Orchestration Workflow

5.1 Agent sequence & routing

`ClinicalWorkflow.run_steps(state)` is an async generator — the single source of truth for the agent sequence — consumed both by the synchronous `POST /orchestration/run` (drains it fully) and the streaming endpoint (yields after each step):

```
1. clinical_intake_agent
   -> if critical fields missing OR missing_information >= 6:
       HALT, determination = "pended"
2. guideline_retrieval_agent
3. risk_stratification_agent
4. criteria_matching_agent
5. determination_agent
6. safety_validation_agent
```

5.2 Streaming protocol

`POST /orchestration/run/stream` returns newline-delimited JSON, one object per agent as it completes, then a final object:

```

{"type": "agent", "agent": "clinical_intake_agent", "status": "success",
 "input": {...}, "output": {...}, "execution_time_ms": 0.03}
... one line per agent ...
{"type": "final", "run_id": 1, "determination": "approved", ...FinalClinicalResponse}

```

Implementation detail with real impact: the embedding call inside `guideline_retrieval_agent` is synchronous, CPU-bound work. Left inline, it fully blocks the `asyncio` event loop — on Windows' default `ProactorEventLoop` this silently prevented any previously-yielded chunk from actually reaching the client until the blocking call returned, turning "streaming" into one buffered burst. Fixed by wrapping the call in `asyncio.to_thread(...)` in `retrieval_agent.py`.

6. API Specification

Method & Path	Purpose
GET /api/v1/health	DB + vector store liveness.
POST /api/v1/patients	Create a synthetic patient.
GET /api/v1/patients	List patients.
GET /api/v1/patients/{id}	Get one patient.
GET /api/v1/guidelines	List ingested policy documents.
POST /api/v1/guidelines/search	Semantic search over the policy corpus (query, top_k).
POST /api/v1/orchestration/run	Run the full pipeline synchronously; returns <code>FinalClinicalResponse</code> .
POST /api/v1/orchestration/run/stream	Same pipeline, NDJSON streamed per-agent.
GET /api/v1/orchestration/runs/{id}	Run summary (status, determination, timestamps).
GET /api/v1/orchestration/runs/{id}/trace	Full per-agent trace for a run.
GET /api/v1/orchestration/runs/{id}/report.pdf	Downloadable PDF rendering of the stored determination + review outcome.
GET /api/v1/review/queue	Runs with <code>review_status = pending_review</code> , oldest first.
GET /api/v1/review/{id}	Full AI response + review state for one run.
POST /api/v1/review/{id}	Submit uphold/override; override requires <code>final_determination</code> + notes.
GET /api/v1/observability/metrics	Aggregated health/volume/outcome/agent/LLM/review metrics. Accepts <code>?period=7d 30d month year all</code> (default all) to scope every section to a trailing window.
GET /api/v1/observability/costs	LLM spend per execution and by call type. Accepts <code>?period=7d 30d month year custom</code> (default 30d; custom requires <code>start_date/end_date</code>) and an optional <code>?model=</code> filter.

6.1 OrchestrationRunRequest (request body)

Field	Type	Notes
patient_id	int	Required.
requested_service	str	Required, min length 1.
service_category	str	Default "other" (imaging / surgery / medication / therapy / dme / other).
diagnosis_codes	list[str]	Default [].
urgency	str	Default "routine" (routine / urgent / emergent).
prior_treatments_tried	list[str]	Default [].
clinical_question	str	Optional free-text additional context.

6.2 SubmitReviewRequest (request body)

Field	Type	Notes
reviewer_name	str	Required, min length 1.
decision	"uphold" "override"	Required.
final_determination	"approved" "denied" "pending"	Required if decision = override; server-validated.
notes	str	Required (non-blank) if decision = override; optional on uphold.

Server-side guards: a run already in review_status = reviewed returns HTTP 400 on a second review attempt; an override without notes or without final_determination returns HTTP 400.

7. RAG Pipeline Detail

7.1 Chunking

Policy .txt files are parsed by "SECTION:" headers (chunker.py), then each section is split into ≤ 600 -character sentence-aligned sub-chunks so individual "Criterion N:" sentences are never truncated mid-sentence.

7.2 Embedding & Indexing

A lazily-loaded, process-cached Sentence-Transformers model (default all-MiniLM-L6-v2) embeds chunks and queries; vectors are L2-normalized and stored in a faiss.IndexFlatIP (inner product on normalized vectors = cosine similarity), with a JSON metadata sidecar in the same order as the index.

7.3 Retrieval

GuidelineRetrievalAgent.run() calls retriever.search() via asyncio.to_thread(...) (see §5.2 note). Results below MIN_RELEVANCE_SCORE = 0.15 are discarded before being handed to the criteria matching agent.

8. LLM Integration (llm_client.py)

generate_determination_rationale() and generate_approval_path_suggestion() are called with the already-decided determination, the full criteria evaluation, and a short patient summary. Both build a prompt, then delegate to a single dispatcher:

```
def _call_llm(prompt, settings) -> tuple[str, LlmUsage]:
    if settings.llm_provider == "openai":
        import openai
        client = openai.OpenAI(api_key=settings.openai_api_key)
        response = client.chat.completions.create(
            model=settings.openai_model,
            max_completion_tokens=settings.llm_max_tokens,
            messages=[{"role": "user", "content": prompt}],
        )
        ... # returns (text, LlmUsage(model, prompt_tokens, completion_tokens))

    import anthropic
    client = anthropic.Anthropic(api_key=settings.anthropic_api_key)
    response = client.messages.create(
        model=settings.anthropic_model,
        max_tokens=settings.llm_max_tokens,
        messages=[{"role": "user", "content": prompt}],
    )
    ... # returns (text, LlmUsage(model, input_tokens, output_tokens))
```

The prompt explicitly instructs the model not to change the determination or invent facts beyond what is listed. On success, the caller returns the drafted text plus an `LlmUsage(model, input_tokens, output_tokens)` taken directly from the active provider's response usage field. On any failure — no API key for the selected provider, SDK import error, network/API error — it returns a deterministic templated rationale built from the same criteria list, sets `usage = None`, and records the failure reason, so the pipeline never fails because of this call.

Setting	Default	Purpose
LLM_PROVIDER	anthropic	Selects which key/SDK is used below — "anthropic" or "openai".
ANTHROPIC_API_KEY	"" (disabled)	Enables the Anthropic path when LLM_PROVIDER=anthropic.
ANTHROPIC_MODEL	claude-haiku-4-5-20251001	Model used for rationale/scenario drafting on Anthropic.
OPENAI_API_KEY	"" (disabled)	Enables the OpenAI path when LLM_PROVIDER=openai.
OPENAI_MODEL	gpt-4o-mini	Model used for rationale/scenario drafting on OpenAI.
LLM_MAX_TOKENS	500	Cap on rationale/scenario length.
LLM_INPUT/OUTPUT_COST_PER_MILLION	1.0 / 5.0	Illustrative \$/1M-token rates used only for the observability cost estimate.

9. Human-in-the-Loop Review Detail

State machine on `orchestration_runs.review_status`:

```

pending_review
  --(POST /review/{id}, decision=uphold)-->
  reviewed (decision=upheld, final_determination = AI's)

pending_review
  --(POST /review/{id}, decision=override)-->
  reviewed (decision=overridden,
            final_determination = reviewer's choice)

reviewed
  --(any further POST)--> 400 Bad Request (no re-review)

```

`review_service.ReviewService` encapsulates this; `ReviewRepository` methods (`list_pending_review`, `submit_review`) are the only writers of these columns.

10. PDF Report Generation (`report_service.py`)

`generate_determination_report(run, patient)` renders `run.final_response_json` (the exact stored response) plus the run's review columns into a ReportLab Platypus document: a summary table, rationale, a criteria-evaluated table, a retrieved-evidence table, safety flags, and a human-review section (reviewer/decision/notes when reviewed, or a not-yet-reviewed notice). This is a pure rendering step — it performs no new computation and cannot diverge from what a reviewer actually saw. Exposed via `GET /orchestration/runs/{id}/report.pdf`.

The platform's standalone design/user documents (this LLD, the HLD, and the User Guide) are generated the same way, from `backend/scripts/generate_platform_docs.py` — a separate ReportLab script, run manually when these documents' content changes, rather than from live run data.

11. Frontend Component Design

Piece	Responsibility
<code>api/client.ts</code>	Typed fetch wrapper; <code>runOrchestrationStream()</code> is an async generator that reads the NDJSON response body incrementally via the Streams API.
<code>agentPipeline.ts</code>	Canonical 6-step definition (mirrors the backend sequence) + pure functions <code>buildInitialPipeline</code> / <code>applyAgentEvent</code> / <code>finalizePipeline</code> driving the live pipeline UI purely from streamed events — no polling.
<code>PatientDetailPage</code>	PA request form; pre-fills diagnosis codes from the patient's documented <code>medical_history</code> (via a small condition-to-ICD-10 lookup), still freely editable; consumes the stream; renders <code>AgentPipeline</code> + <code>AgentTraceView</code> live, then <code>AssessmentResult</code> on completion.
<code>AssessmentResult</code>	Renders a <code>FinalClinicalResponse</code> : determination badge, rationale (+ LLM/template provenance), criteria table, evidence, safety flags, and the PDF download link.
<code>ReviewQueuePage</code> / <code>ReviewDetailPage</code>	Lists <code>pending_review</code> runs; detail page reuses <code>AssessmentResult</code> and adds the uphold/override form with client + server-side validation.
<code>ObservabilityPage</code> + charts/	Fetches <code>/observability/metrics</code> with a page-level period selector (7d/30d/month/year/all) that re-queries the whole dashboard; separately fetches <code>/observability/costs</code> with its own 7d/30d/month/year/custom filter for the cost trend chart and per-run cost table. Renders health pills, stat tiles, two hand-built stacked-bar charts (status-colored part-to-whole), and Recharts grouped/stacked/line charts (agent latency on a log scale, agent reliability, cost trend) with a shared, palette-validated color module and a shared tooltip component.

12. Error Handling

Exception	HTTP status	Trigger
PatientNotFoundError	404	Unknown patient_id.
OrchestrationRunNotFoundError	404	Unknown run_id.
InvalidPayloadError	400	Duplicate patient identifier; already-reviewed run; override missing notes/final_determination; report requested for an incomplete run; unknown observability period.
VectorStoreUnavailableError	503 (or caught by BaseAgent → trace status="error")	FAISS index missing/unreadable.
EmbeddingModelUnavailableError	503	Model failed to load.
Pydantic validation error	422	Malformed request body (e.g. blank requested_service).

All AppError subclasses are mapped to a JSON body of the shape {error, message, requires_clinician_review} by a single FastAPI exception handler in main.py.

13. Testing Strategy

65 backend tests (pytest + pytest-asyncio + FastAPI TestClient), fully offline via a deterministic hashed bag-of-words embedding stand-in (tests/fake_embeddings.py):

- Patients: CRUD + validation.
- Guidelines: ingestion + search, including a no-relevant-result path.
- Each agent in isolation, including the two regression tests for the BMI-alternative-band and negated-exclusion fixes.
- Full streaming workflow: event ordering, halted/pended path, agent-failure resilience.
- Human-in-the-loop: queue listing, uphold, override, double-review rejection, validation guards.
- PDF report: generation for a completed run, a reviewed run, and 404 for a missing run.
- LLM provider selection (Anthropic vs. OpenAI) and graceful template fallback on missing key/SDK/call failure.
- Observability metrics service: aggregation correctness, empty-history edge case, period-filter scoping (including exclusion of out-of-range runs), and cost-endpoint named/custom period handling.

Frontend correctness is verified via tsc --noEmit, a full Vite production build, and — for UI-affecting changes in this project's history — live runs against the actual dev servers rather than mocks.